

Submission 27	Help	Conference	News	EasyChair
---------------	------	------------	------	-----------

ARITH2026 Submission 27

Submission information updates are disabled. If you need to update your submission, contact ARITH2026 chairs.

For all questions related to your submission you should contact the conference organizers. [Click here to see information about this conference.](#)

All **reviews sent to you** can be found at the bottom of this page.

Submission 27	
Title	High Throughput Large Integer Multipliers Built on Optimized FPGA DSP Block Bases
Paper	 (Feb 14, 13:06)
Suppl	
Track	ARITH2026
Author keywords	Arithmetic Multipliers DSP blocks
EasyChair keyphrases	dsp blocks (160), large multipliers (130), large integer multiplication (110), higher frequency (90), bit multiplier (80), bit base multiplier (79), overshoot tiling (70), base sb karat sb karat (69), n bit multiplier (63), usage and latency (63), multi group strategy (63), target constrained tiling (63), target size (60), architecture aware (60), schoolbook decomposition (60), large sizes (60), design of integrated circuits and systems (51), high throughput (50), partial products (50), computer arithmetic arith (47), square base multiplier (47), aided design of integrated circuits (46), integrated circuits and systems pp (46), dsp block (45), bit base (43), low frequency (40), shift amount (40), lut usage (40), decomposition schemes (40), normalized area (40), shift and add logic (40), design of large multipliers (40), luts ffs (40), fewer luts (40), fewer dsps (40), bit signed base multipliers (40), slightly increased lut usage (40), combinational logic segment size (40), fast pipelined combinational logic (40), non hierarchical rectangular regions (40)
Abstract	Large integer multiplication is critical in cryptographic applications such as Fully Homomorphic Encryption (FHE) and RSA where it represents the main step in computing the modular multiplication used to generate keys. Most previous work building such multipliers on FPGAs has overlooked the importance of considering the underlying architecture of the Digital Signal Processing (DSP) Blocks present in modern FPGAs. These DSP blocks have an irregular input size that must be tiled carefully to create larger multipliers, and have complex pipelining configurations, which must be respected to achieve high frequency. We show how to build efficient base multipliers that can then be tiled to large sizes efficiently through automated architecture-aware tiling and fast pipelined datapath generation, while maintaining resource efficiency and high throughput. We demonstrate full automation of large multiplier generation, achieving 56% higher frequency while using about 17% fewer DSP blocks compared to prior approaches.
Submitted	Jan 22, 08:08
Last update	Feb 14, 10:51
Author conflicts	none

Authors						
First name	Last name	Email	Country	Affiliation	Web page	Corresponding?
Mahmoud	Elshenawy	mahmoud.elshenawy@kaust.edu.sa	Saudi Arabia	King Abdullah University of Science and Technology		✓
Zhiru	Zhang	zhiruz@cornell.edu	United States	Cornell University		✓
Suhaib A.	Fahmy	suhaib.fahmy@kaust.edu.sa	Saudi Arabia	King Abdullah	https://accl.kaust.edu.sa	✓

Reviews

Review 1	
Overall evaluation	The paper presents an approach for constructing an efficient, architecture-aware base multiplier that leverages FPGA DSP blocks effectively. The topic is clearly relevant to the conference, and the paper is generally well-structured and easy to follow. However, a primary concern is the absence of a detailed discussion regarding the correctness of the proposed approach. The evaluation focuses on performance comparisons with existing implementations, without addressing functional correctness. This aspect is fundamental and should not be overlooked, particularly for a contribution of this nature. In addition, some minor issues could be improved: – Section III.A: It would be beneficial to include a discussion of the trade-offs involved in reducing DSP usage at the expense of increased LUT utilization. – Section III.E: There appears to be a formatting issue with spacing after the section title. Furthermore, including a figure or block diagram would significantly enhance clarity and aid in understanding the described flow.
Review 2	
Overall evaluation	This paper introduces a scientifically rigorous, architecture-aware hierarchical tiling algorithm for constructing large integer multipliers on FPGAs. By framing irregular DSP grids as a mathematical decomposition problem, it effectively optimizes Karatsuba and Schoolbook base scaling. pros: – Formalizes mathematical minimization of asymmetric DSP block waste. – Advances hierarchical algorithmic tiling over rigid template matching. – Pareto analysis robustly justifies optimal square base selections. cons: – Lacks empirical comparison against NTT-based multipliers for >1024-bits. – No theoretical proofs bounding the heuristic's optimal tiling efficiency. – Scientific evaluation ignores dynamic power consumption impacts entirely. – Architecture focus heavily overfits to specific AMD DSP geometries Overall comments: The paper presents a strong architectural methodology but requires scientific rigor and structural polish. To mitigate current cons, the authors must compare their >1024-bit designs against SOTA NTT/FFT multipliers, rather than relying solely on HLS baselines. Scientifically, the manuscript needs a formal mathematical proof bounding the heuristic's DSP waste (remaining area in under-fit components) against the theoretical minimum, alongside a comprehensive dynamic power analysis.
Review 3	
Overall evaluation	This paper deals with the design of large integer multipliers (160--1024 bits) for AMD FPGAs, with a focus on high frequency. The sizes correspond to typical asymmetric cryptographic applications (elliptic curve cryptography or RSA). The authors claim in the introduction that an open-source tool, not available at the time of the review, will be released with the paper The paper is well written, provides good results, but the comparison section must be improved and a validation section must be added. *** Brief Summary *** The approach of the authors is to build "square" smaller multipliers ($N \times N$) and combine them with Schoolbook or Karatsuba to perform larger multiplications (by 2×2 or 3×3 grids). The small $N \times N$ multipliers are called base multipliers. Base multipliers are obtained through a tiling method of the 25×18 or 27×18 DSP blocks from AMD FPGAs (25×18 for DSP48E1, 27×18 for DSP48E2 as in the proposed implementation results). This method is a variant of [14], but takes into account the cascading properties of these DSP blocks. Fig. 3 illustrates these properties, and Fig. 5 shows 2 strategies to use them to build base multipliers. With the presented method, base multipliers are 44 and 78 bits (larger ones do not give better results for the target sizes). Compared to the AMD IP 64-bits multiplier obtained with Vivado, the tiled 78×78 base multiplier takes 4 fewer cycles and 2 fewer DSPs. Then the base multipliers (43 and 77 for unsigned base multipliers) are tiled to

obtain the target size multiplier (for instance 1024 bits, see Fig. 7 for 2 different strategies).

Finally, the authors provide implementation results on AMD Alveo U280 FPGA for 160, 256, 384, 512 and 1024 bits multipliers, and compare them with previous state-of-the-art works.

*** Main Comments ***

My main complaints are about the application context, the comparisons and the lack of verification process presented.

For the application context, authors should remove ECC on 160 bits because it is discouraged now.

Then, authors on page 6 wrote "Our method achieves significantly higher frequency (and hence throughput) than previous work", but, at the application level, the equivalence frequency/throughput is not necessarily true. If an RSA exponentiation is computed, for instance, each iteration (which computes 1 multiplication and 1 square) depends on the result of the previous one, thus one cannot fill the pipeline of the multiplier (in ECC one has more multiplications, but not enough to completely fill the pipeline). Thus, from the results in Table II, one can use Impress1[7] to get each multiplication in 23 cycles at 250 MHz (with 810 DSPs) against 100 cycles at 458 MHz (also with 810 DSPs) for the proposition, in this case [7] gives a better throughput than the current paper in term of RSA exponentiations per second.

The authors should also recall the FPGAs used in state-of-the-art results (Table II), for instance [8] results are given for a Kintex 7 FPGA (with 25×18 DSP48E1 blocks and not DSP48E2, with older technology so the frequencies can be hard to compare directly).

The authors present results for Versal VCK190, and wrote that "the DSP58-based multiplier significantly reduces DSP usage and improves frequency, at the cost of a slight increase in LUTs".

However, DSP58 is larger than DSP48 (it has 27×24 bit multipliers, 58-bit accumulator...) thus it is hard to directly compare the results to those obtained with DSP48E1 or E2.

Compared to the 27×18 of DSP48E2, the DSP58 offers 27×24 multipliers (both inputs are of almost the same size). One can imagine that it would change the tiling strategies to have both inputs of almost the same size, it would have been interesting to have a discussion in the paper about it.

The "normalized" area metric ($= \text{LUTs} + 126 \times \text{DSPs}$), based on the total number of LUTs and DSPs is quite strange because it is very dependent on the target FPGA (it is not specified if, for each reference, a different formula has been used to take into account the other FPGAs characteristics). Depending on the FPGA family and the applications targeted, the ratio LUTs/DSPs can change a lot. The figure 10 and the normalized area metric could be removed to discuss with more details on the differences mentioned above.

Some papers are cited in the related works section (sec. V), but not presented in the table. Even if these works cannot give results for 1024-bit integers or with lower frequency, authors should include more of them in the table. As an example, authors of [14] mention a 192 bits multiplier (on Virtex 5) with 8 DSP48E1 (and 426 slices) which takes 15 cycles at 250 MHz, and a 256 bits multiplier (with schoolbook) with 11 DSP (800 slices) which takes 19 cycles (at 200 MHz). The frequency is 3 or 4 times lower than the proposition, but it uses 8 times fewer resources and 2 times fewer clock cycles. One could put multiple operators of [14] in parallel, increasing the size to match with the proposed one, but increasing at the same time its throughput.

The goal is not exactly the same between [14] and the current paper, but at the end they both provide ECC size multipliers. The paper would have benefited from a wider range of results in the comparison section. In the current paper, only 1024-bit state-of-the-art implementation results are reported in Table 2.

Finally, there is no real discussion about the functional verification of the computation (except on page 6 where "testbenches for functional verification" are briefly mentioned, without details).

For this kind of work on operator generators, we expect a section dedicated to the presentation of the type and the number of tests performed, on what kind of inputs, configurations etc.

*** Figures ***

Figures are nice, but some of them have too much information presented making them hard to read, and some of them are too small compared to the quantity of information presented.

Fig. 2, page 2: P1,P2,P3 are too small (cells containing 1 are not readable without a x3 zoom)

Fig. 5 is nice, but very small in the printed version, the legend of Fig. 6 is very small too.

In Fig. 6, the right-most figure can be misleading, as the y-axis does not start at zero. This visually amplifies the differences (which seem to be at most 10%).

Fig. 8 and Fig. 10 are hard to read due to the combination of multiple colors/symbols/inner filling. It shows that

authors tested many configurations however, a simpler representation focusing on the most relevant results would improve clarity.

*** Typo ***

page 1: "1024-bit-bit design"

*** Conclusion ***

This paper is mainly well written, presents nice figures (but some of them are too small), and the results provide higher throughput than state-of-the-art multipliers.

However, the comparison section should have been more detailed, with more detailed state-of-the-art results, and more details on the verification process of the generator and the produced multipliers.